

TERRAFORM TASK - 04

1. Execute the script shown in the video.

Version Constraints:

Version constraints in Terraform are string literals used to restrict which versions of the Terraform CLI (Core), Providers (like AWS, Azure, GCP), and Modules are allowed to execute your configuration. They act as a safety net to prevent unexpected infrastructure failures caused by breaking changes in newer software updates.

The Three Targets of Version Constraints

Version constraints are typically applied to three distinct areas within your infrastructure code:

1. **Terraform Core CLI (required_version):** Restricts the execution engine version.
2. **Providers (required_providers):** Restricts the versions of cloud plugins downloaded during initialization.
3. **Modules (version):** Restricts the version of external, reusable architecture packages pulled from public or private registries.

Here is an example showing all three in action:

hcl

```
terraform {
  required_version = ">= 1.5.0, < 2.0.0" # 1. Core constraint

  required_providers {
    aws = {
      source = "hashicorp/aws"

      version = "~> 5.0" # 2. Provider constraint
    }
  }
}
```

```

}

}

module "vpc" {

    source = "terraform-aws-modules/vpc/aws"

    version = "5.0.0" # 3. Module
    constraint

}

```

Types of Version Constraints and Their Definitions

Terraform identifies specific allowed versions using **constraint operators**. The official [HashiCorp Terraform Documentation](#) defines them as follows:

1. Exact Match (= or no operator)

- **Definition:** Dictates that only one specific version string is acceptable.
- **Example:** version = "5.67.1" or version = "= 5.67.1"
- **Behavior:** Terraform will only download or run version 5.67.1. Any higher or lower version triggers an error. This is highly recommended for production modules. [[1](#), [2](#), [3](#), [4](#), [5](#)]

2. Not Equal (!=)

- **Definition:** Explicitly blocks one particular version while allowing any others that meet adjacent requirements.
- **Example:** version = ">= 1.5.0, != 1.5.1"
- **Behavior:** It avoids a known bug or problematic patch (like 1.5.1) but allows versions like 1.5.0 or 1.5.2. [[1](#)]

3. Comparison Operators (>, >=, <, <=)

- **Definition:** Establishes lower bounds, upper bounds, or ranges for version matching. [[1](#), [2](#)]
- **Examples:**
 - version = ">= 1.7.5" (Allows any version equal to or newer than 1.7.5).

- version = "< 2.0.0" (Allows any version strictly older than 2.0.0). [[1](#)]

4. Compound / Logical AND Constraints (,)

- **Definition:** Combines multiple comparison operators using a comma to create an explicit range.
- **Example:** version = ">= 1.2.0, < 2.0.0"
- **Behavior:** The comma acts as an **AND** operator. This allows any version from 1.2.0 up to—but not including—2.0.0. [[1](#), [2](#)]

5. Pessimistic Constraint Operator (~>)

- **Definition:** Also known as the "twiddle-wakka," this operator sets a minimum boundary but allows only the **rightmost version component** to increment. It protects you against major or minor breaking updates while automatically absorbing safe bug fixes. [[1](#), [2](#)]
- **Behavior depends on the number of digits you specify:**
 - **Three digits (~> X.Y.Z):** Only the *patch* version can change.
 - *Example:* ~> 1.5.0 translates to >= 1.5.0, < 1.6.0. It allows 1.5.1 and 1.5.9, but blocks 1.6.0.
 - **Two digits (~> X.Y):** The *minor* version can change.
 - *Example:* ~> 5.0 translates to >= 5.0.0, < 6.0.0. It allows 5.1.0 and 5.50.0, but blocks 6.0.0. [[1](#), [2](#), [3](#)]

← → ↻ 🌐 registry.terraform.io/providers/hashicorp/local/latest

 Terraform | Registry

Browse

Publish

Sign In

🔍 Search providers, modules, run tasks, and policies...

/

Providers / hashicorp / local / v2.9.0

 **local** Official

hashicorp/local

Used to manage local resources, such as creating files

Downloads: 1.2B This week: 6.3M Versions: 27 Source code: hashicorp/local Published: May 13, 2026 Published by: HashiCorp

Utility

Overview

Documentation

📦 Top downloaded local modules

Modules are self-contained packages of Terraform configurations that are managed as a group.

Showing 1 - 4 of 950 available modules

 **terraform-aws-modules / eks**
Terraform module to create Amazon Elastic Kubernetes (EKS) resources ua
🕒 24 days ago 📄 176.9M

 **terraform-aws-modules / lambda**
Terraform module, which takes care of a lot of AWS Lambda/serverless tasks (build dependencies, packages,...
🕒 4 months ago 📄 137.2M

Version 2.9.0 (latest)

🔍 ✕

✓ Version 2.9.0 (latest)

Version 2.8.0

Version 2.7.0

Version 2.6.2

Version 2.6.1

📄 View Source

📄 Provider Downloads

All versions

Downloads this week

6.3M

Downloads this month

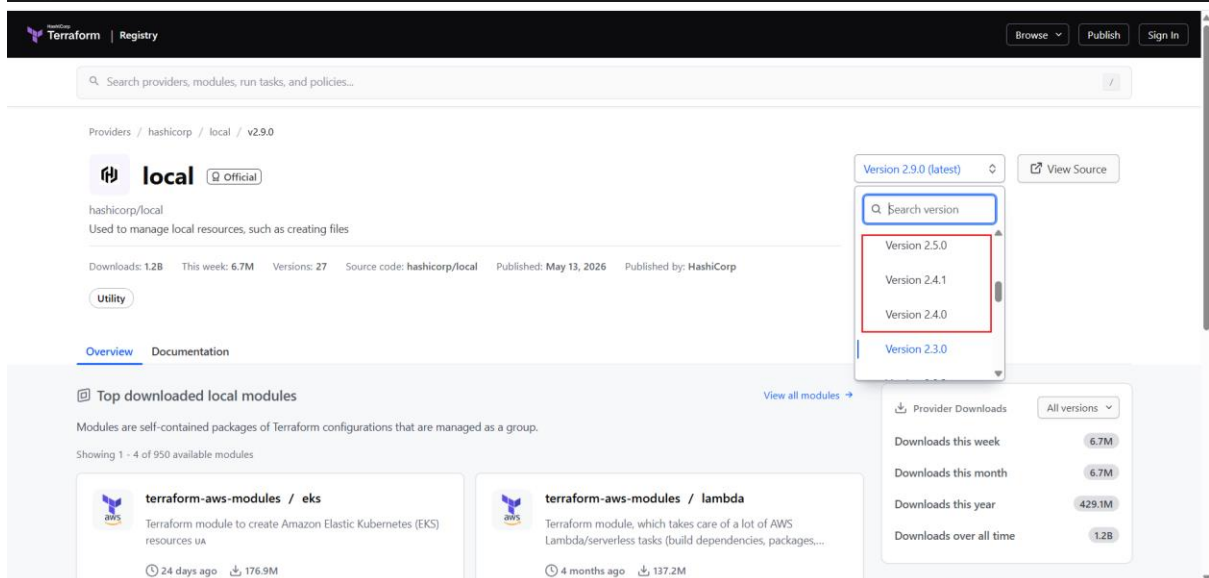
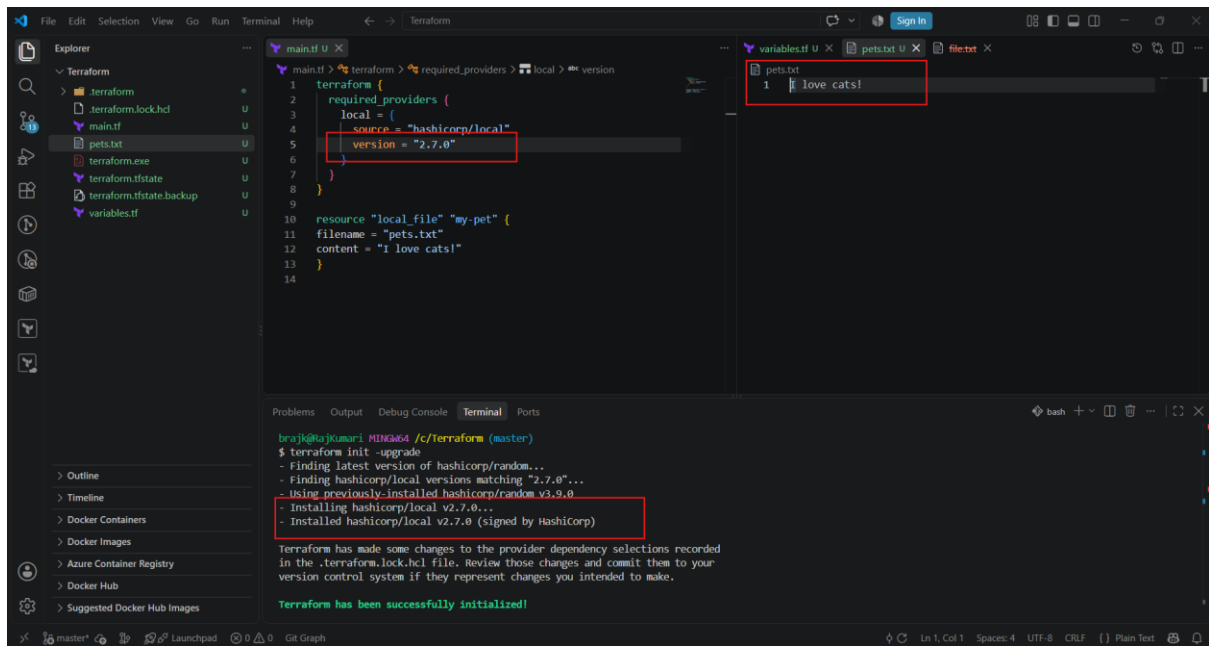
6.3M

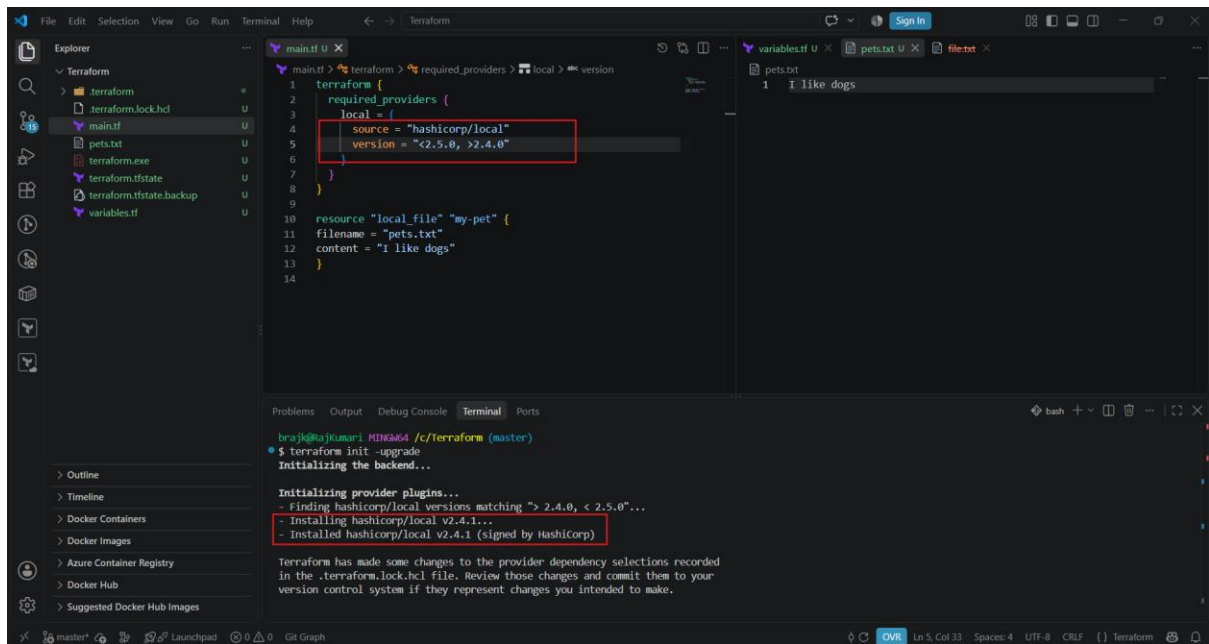
Downloads this year

428.8M

Downloads over all time

1.2B





Data Source: A data source in Terraform is used to read or retrieve information about existing infrastructure or external data without creating or managing that resource.

Example:

```
data "aws_ami" "amazon_linux" {
  most_recent = true

  owners    = ["amazon"]
}
```

Data Source vs Resource

Resource

Creates infrastructure

Uses resource block

Can modify/delete resources

Data Source

Reads existing infrastructure

Uses data block

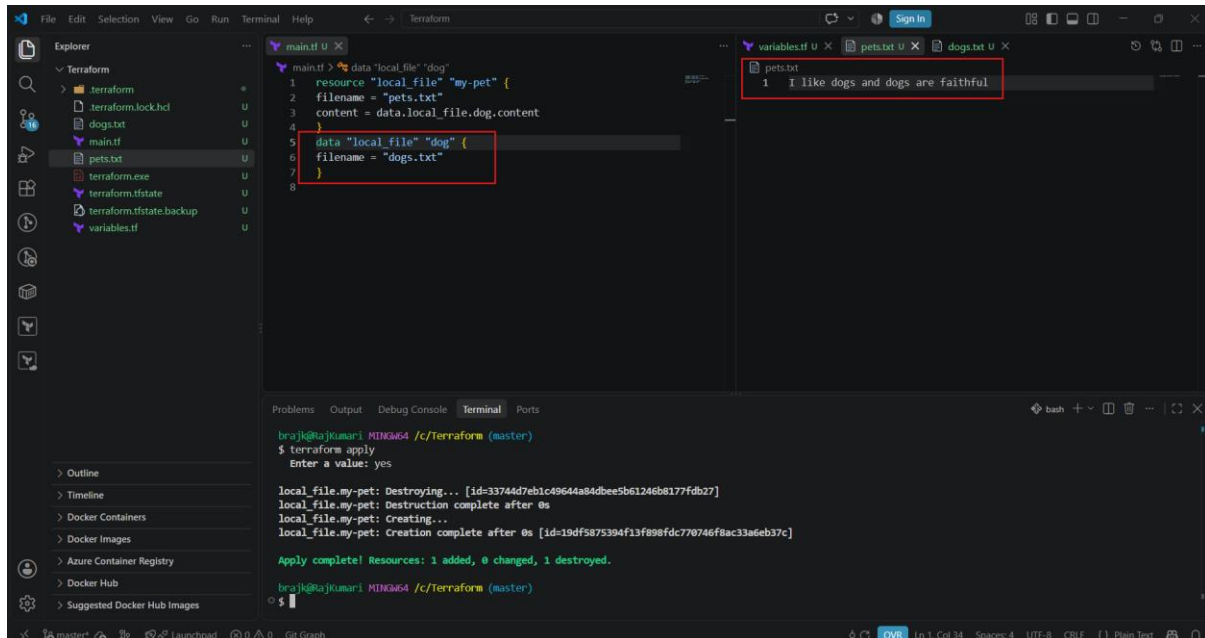
Read-only

Resource

Stored in Terraform state as managed objects

Data Source

Read values for use during planning/apply



The screenshot shows the VS Code editor with the Terraform project structure in the Explorer. The main.tf file contains a resource definition for 'local_file' 'my-pet' and a data source 'local_file' 'dog'. The pets.txt file contains the text 'I like dogs and dogs are faithful'. The terminal shows the output of the 'terraform apply' command, indicating that the resource was created successfully.

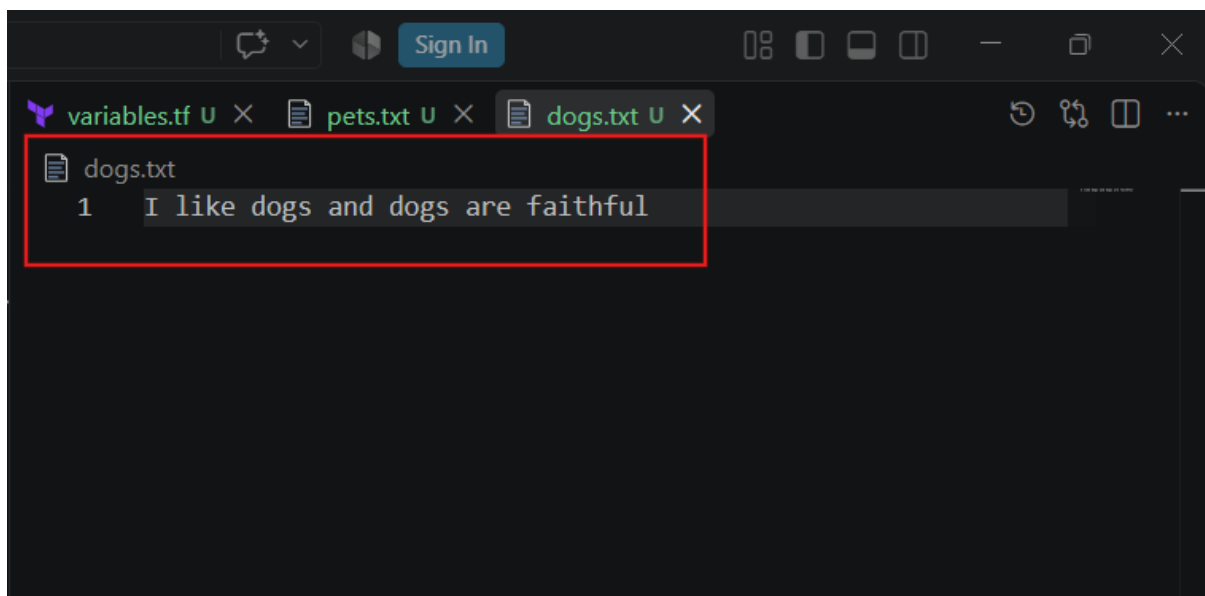
```
main.tf
1 resource "local_file" "my-pet" {
2   filename = "pets.txt"
3   content = data.local_file.dog.content
4 }
5 data "local_file" "dog" {
6   filename = "dogs.txt"
7 }
8
```

```
pets.txt
1 I like dogs and dogs are faithful
```

```
terminal
brajk@rajKumari-MINGW64 /c/Terraform (master)
$ terraform apply
Enter a value: yes

local_file.my-pet: Destroying... [id=33744d7eb1c49644a84dbec5b61246b8177fdb37]
local_file.my-pet: Destruction complete after 0s
local_file.my-pet: Creating...
local_file.my-pet: Creation complete after 0s [id=19df5875394f13f898fdc770746f8ac33a6eb37c]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
brajk@rajKumari-MINGW64 /c/Terraform (master)
```



The screenshot shows the VS Code editor with the dogs.txt file open. The file contains the text 'I like dogs and dogs are faithful'.

```
dogs.txt
1 I like dogs and dogs are faithful
```

Meta-Arguments: Meta-arguments are special arguments provided by Terraform that control how resources or modules are created, managed, and processed, rather than defining the actual properties of the resource.

Common Terraform meta-arguments are:

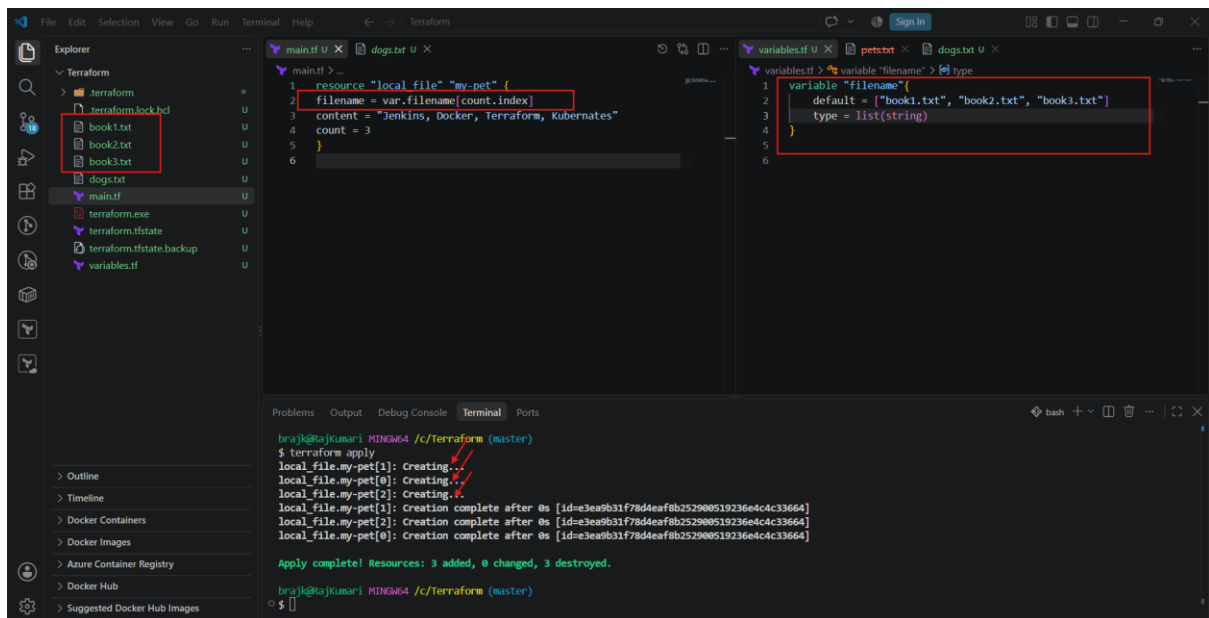
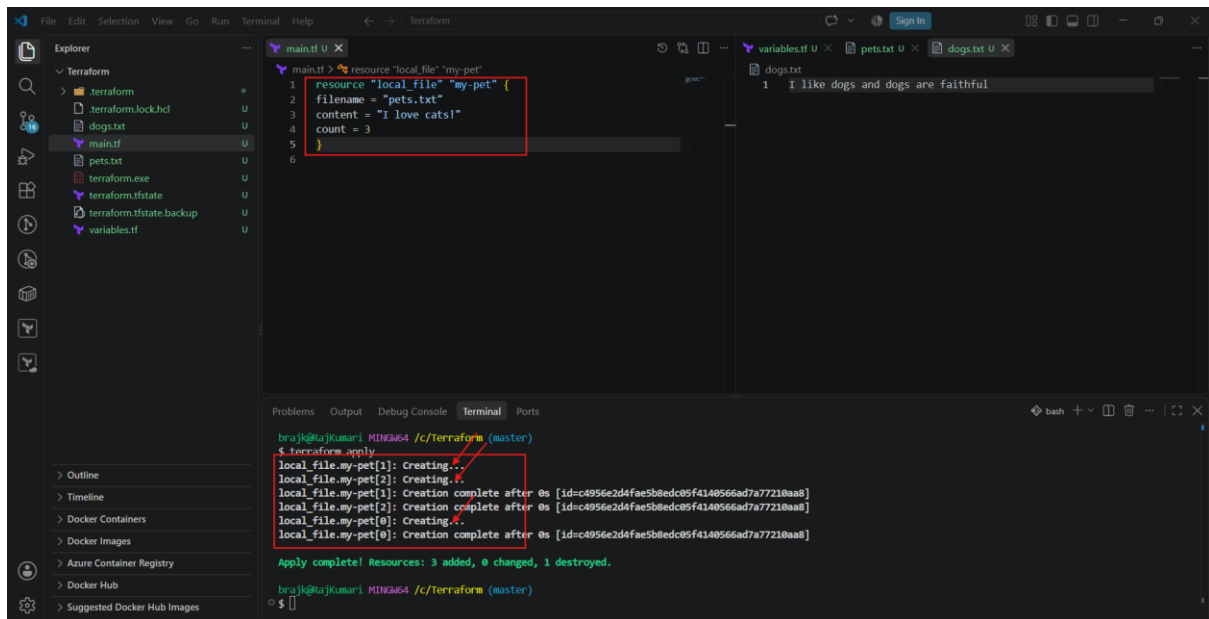
Meta-argument	Purpose
count	Creates multiple instances based on a number
for_each	Creates multiple instances based on a map or set
depends_on	Defines explicit dependencies
provider	Selects which provider configuration to use
lifecycle	Controls resource lifecycle behavior

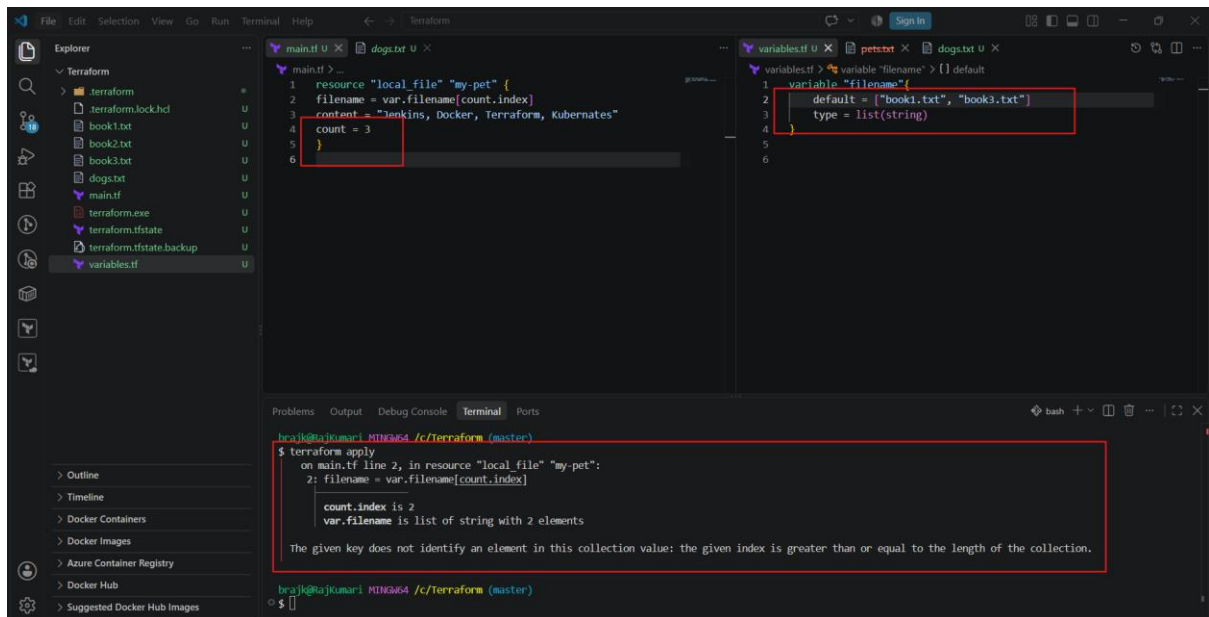
Example of count:

=====

If we use count as 3 then it will create 3 files with pet[0],pet[1],pet[2]

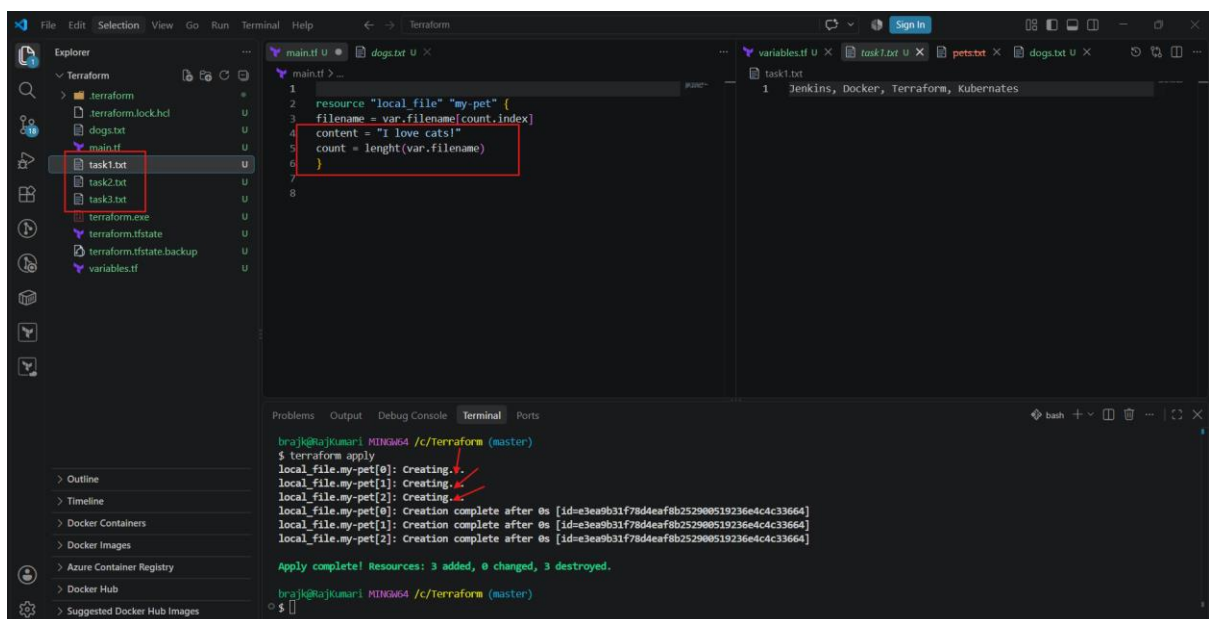
```
resource "local_file" "my-pet" {  
  filename = "pets.txt"  
  content = "I love cats!"  
  count = 3  
}
```

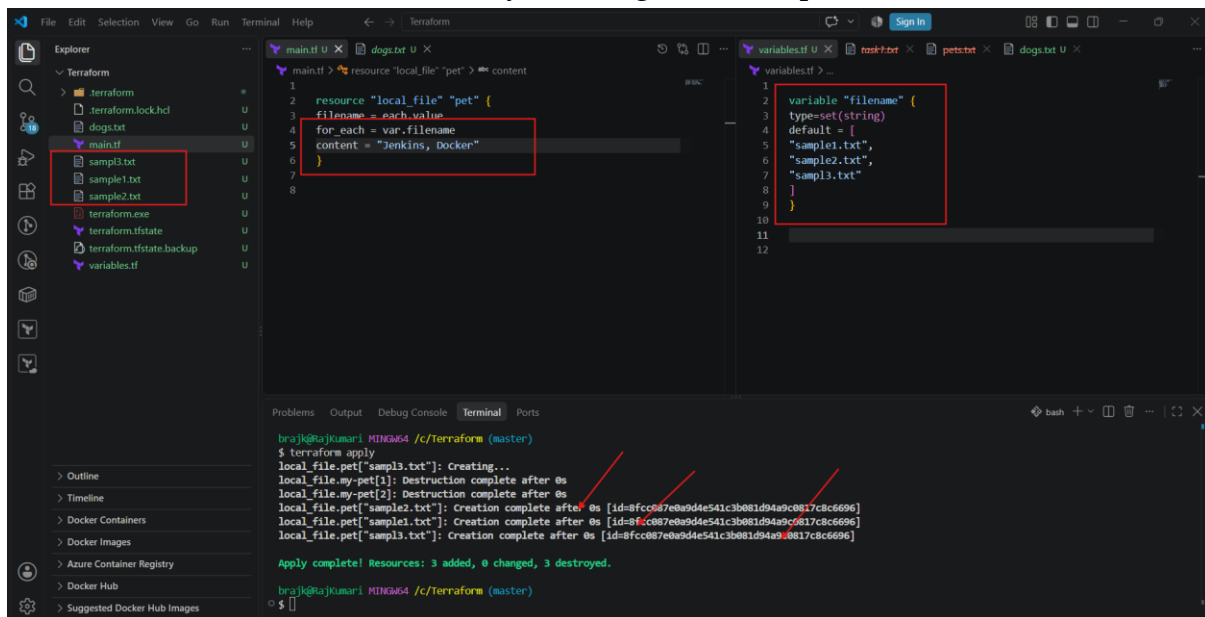


Still we have problem in the above configuration, if in future the list of variables then we need to change the count value manually.

to avoid this we can use the inbuilt length function in terraform.



For_each: for_each is a meta-argument in Terraform that allows us to create multiple instances of a resource or module by iterating over a map or set of values



The screenshot shows a VS Code editor with the following components:

- Explorer:** A file tree on the left showing a project structure with files like `main.tf`, `sample1.txt`, `sample2.txt`, `sample3.txt`, `dogs.txt`, `terraform.lock.hcl`, `terraform.exe`, `terraform.tfstate`, `terraform.tfstate.backup`, and `variables.tf`. The `sample1.txt`, `sample2.txt`, and `sample3.txt` files are highlighted with a red box.
- main.tf:** The main configuration file in the center, showing a `resource "local_file" "pet"` block with a `for_each` loop. The `for_each` loop is highlighted with a red box. The code is:

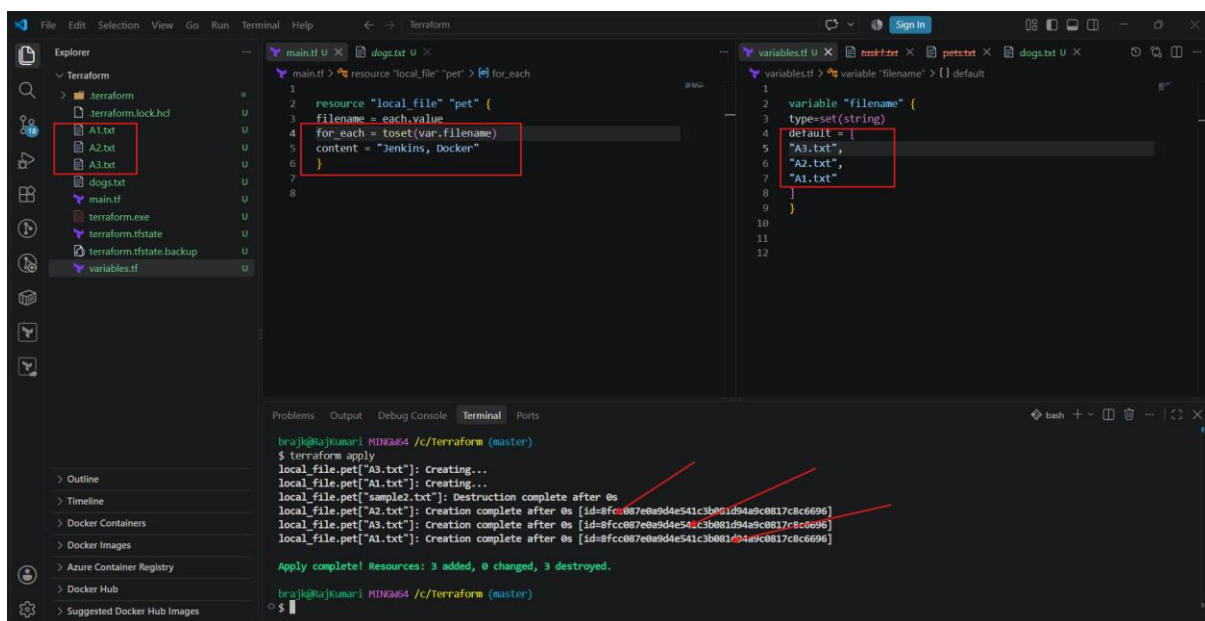
```
1 resource "local_file" "pet" {
2   filename = each.value
3   for_each = var.filename
4   content = "Jenkins, Docker"
5 }
6
7
8
```
- variables.tf:** The variables file on the right, showing a `variable "filename"` block. The `default` list is highlighted with a red box. The code is:

```
1 variable "filename" {
2   type = set(string)
3   default = [
4     "sample1.txt",
5     "sample2.txt",
6     "sample3.txt"
7   ]
8 }
9
10
11
12
```
- Terminal:** The bottom panel shows the output of the `terraform apply` command. It shows the creation of three `local_file` resources: `local_file.pet["sample1.txt"]`, `local_file.pet["sample2.txt"]`, and `local_file.pet["sample3.txt"]`. The output is highlighted with a red box. The terminal text is:

```
brajk@rajKumari-MINGW64 /c/Terraform (master)
$ terraform apply
local_file.pet["sample1.txt"]: Creating...
local_file.pet["sample1.txt"]: Creation complete after 0s [id=8fcc087e0a9d4e541c3b081d94a9c0817c8c696]
local_file.pet["sample2.txt"]: Creation complete after 0s [id=8fcc087e0a9d4e541c3b081d94a9c0817c8c696]
local_file.pet["sample3.txt"]: Creation complete after 0s [id=8fcc087e0a9d4e541c3b081d94a9c0817c8c696]

Apply complete! Resources: 3 added, 0 changed, 3 destroyed.

brajk@rajKumari-MINGW64 /c/Terraform (master)
$
```



The screenshot shows a VS Code editor with the following components:

- Explorer:** A file tree on the left showing a project structure with files like `main.tf`, `A1.txt`, `A2.txt`, `A3.txt`, `dogs.txt`, `terraform.lock.hcl`, `terraform.exe`, `terraform.tfstate`, `terraform.tfstate.backup`, and `variables.tf`. The `A1.txt`, `A2.txt`, and `A3.txt` files are highlighted with a red box.
- main.tf:** The main configuration file in the center, showing a `resource "local_file" "pet"` block with a `for_each` loop. The `for_each` loop is highlighted with a red box. The code is:

```
1 resource "local_file" "pet" {
2   filename = each.value
3   for_each = toset(var.filename)
4   content = "Jenkins, Docker"
5 }
6
7
8
```
- variables.tf:** The variables file on the right, showing a `variable "filename"` block. The `default` list is highlighted with a red box. The code is:

```
1 variable "filename" {
2   type = set(string)
3   default = [
4     "A3.txt",
5     "A2.txt",
6     "A1.txt"
7   ]
8 }
9
10
11
12
```
- Terminal:** The bottom panel shows the output of the `terraform apply` command. It shows the creation of three `local_file` resources: `local_file.pet["A3.txt"]`, `local_file.pet["A2.txt"]`, and `local_file.pet["A1.txt"]`. The output is highlighted with a red box. The terminal text is:

```
brajk@rajKumari-MINGW64 /c/Terraform (master)
$ terraform apply
local_file.pet["A3.txt"]: Creating...
local_file.pet["A3.txt"]: Creation complete after 0s [id=8fcc087e0a9d4e541c3b081d94a9c0817c8c696]
local_file.pet["A2.txt"]: Creation complete after 0s [id=8fcc087e0a9d4e541c3b081d94a9c0817c8c696]
local_file.pet["A1.txt"]: Creation complete after 0s [id=8fcc087e0a9d4e541c3b081d94a9c0817c8c696]

Apply complete! Resources: 3 added, 0 changed, 3 destroyed.

brajk@rajKumari-MINGW64 /c/Terraform (master)
$
```

2. Integrate **Terraform in Jenkins** using the **Terraform plugin**.

GitHub → Jenkins → Terraform Plugin → Terraform → AWS

One important point first: the Jenkins **Terraform plugin** is an older plugin (current listed version 1.0.10), and its documented configuration is primarily a **build wrapper**. For a modern Jenkins pipeline, you can still configure Terraform through **Manage Jenkins → Tools** and invoke Terraform from a Jenkinsfile. This is generally easier to maintain.

1. Overall architecture

Developer

|

| git push

v

GitHub Repository

|

v

Jenkins

|

| Terraform Plugin / Terraform installation

v

Terraform

|

| AWS credentials

v

AWS

|

+---- EC2

+---- VPC

+---- Security Group

+---- S3

+---- etc.

The important distinction is:

- **Jenkins Terraform plugin** → helps Jenkins integrate with Terraform.
- **Terraform AWS provider** → allows Terraform to create/manage AWS resources.

Terraform itself uses provider plugins to communicate with services such as AWS.

Step 2: Install Terraform on the Jenkins server

SSH into your Jenkins EC2 instance.

Check whether Terraform is already installed:

```
terraform version
```

If you get something like:

```
Terraform v1.x.x
```

you can proceed.

If not, install Terraform.

For example, on Amazon Linux:

```
sudo dnf update -y
```

Then install Terraform using HashiCorp's repository/package instructions appropriate for your Amazon Linux version.

After installation:

```
terraform version
```

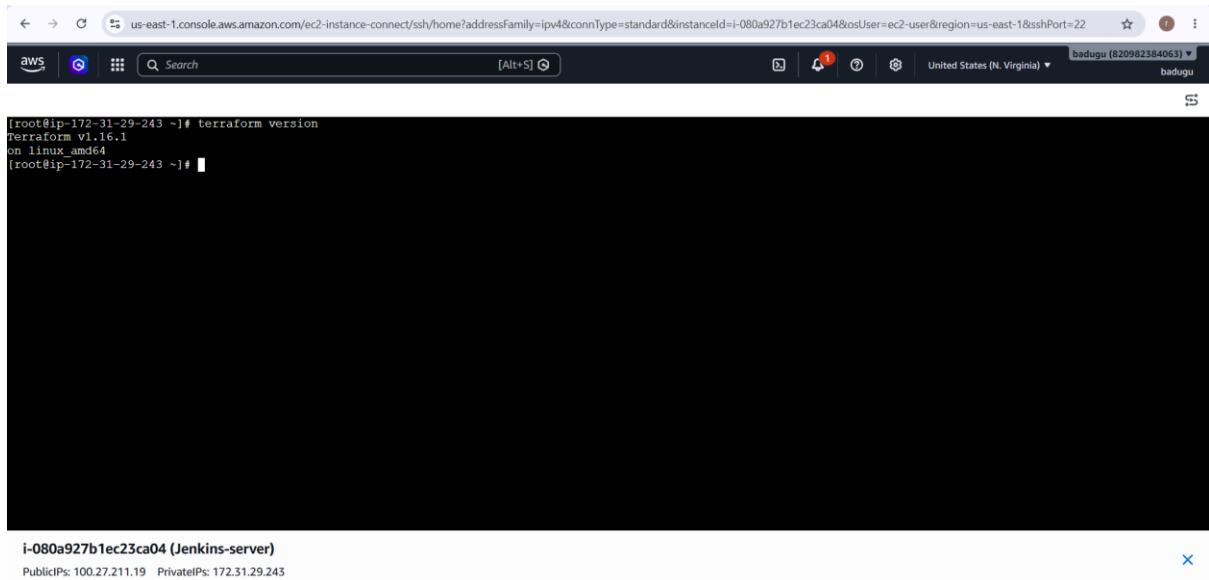
You should see:

```
Terraform v1.x.x
```

```
on linux_amd64
```

Why this is important

Jenkins ultimately needs access to the terraform executable on the machine/agent where the build runs.



The screenshot shows an AWS Management Console terminal window. The terminal output is as follows:

```
[root@ip-172-31-29-243 ~]# terraform version
terraform v1.16.1
on linux_amd64
[root@ip-172-31-29-243 ~]#
```

Below the terminal window, the instance details are visible: **i-080a927b1ec23ca04 (Jenkins-server)**, with PublicIPs: 100.27.211.19 and PrivateIPs: 172.31.29.243.

Step 3: Install the Jenkins Terraform plugin

Go to:

Jenkins Dashboard



Manage Jenkins



Plugins



Available plugins

Search:

Terraform

Install:

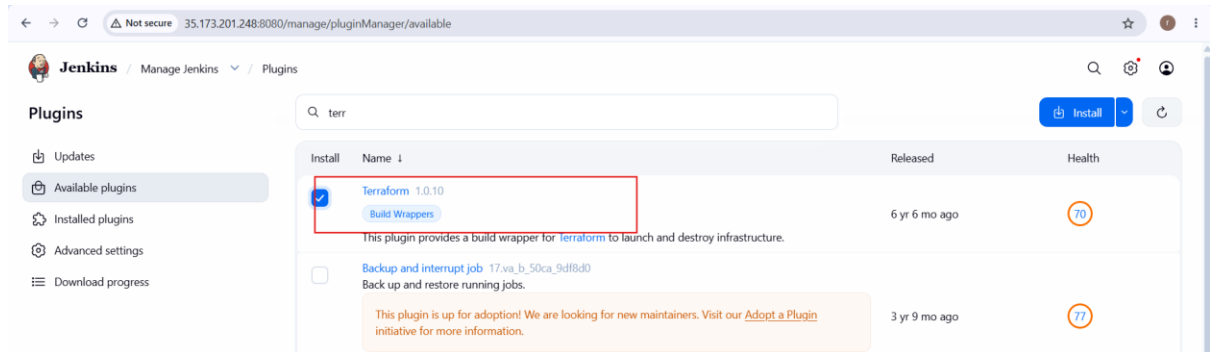
Terraform

The official Jenkins plugin page identifies the plugin ID as:

terraform

and documents installation through the Jenkins plugin manager/CLI.

After installation, restart Jenkins if Jenkins asks you to.



Step 4: Configure Terraform in Jenkins

Go to:

Jenkins Dashboard



Manage Jenkins



Tools

Look for:

Terraform installations

Click:

Add Terraform

Give it a name, for example:

terraform

or:

Terraform-1.13

You can configure the Terraform executable/installation here.

For example:

Name:

terraform

Install automatically:

✓

Or, if Terraform is already installed on your Jenkins server, configure Jenkins to use the existing Terraform installation.

Then click:

Save

The screenshot shows the Jenkins web interface at the URL `35.173.201.248:8080/manage/configureTools/`. The page title is "Jenkins" and the breadcrumb is "Manage Jenkins > Tools". There is a search bar, a settings gear icon, and a user profile icon in the top right. Below the header, there is a button "+ Add Maven". The main section is titled "Terraform installations" and contains a button "+ Add Terraform". Below this, there is a form for configuring a Terraform installation. The form has a title "Terraform" with a close button. It contains a "Name" field with the value "terraform". Below that is a checkbox "Install automatically" which is checked. Underneath the checkbox is a section titled "Install from bintray.com" with a close button. This section contains a "Version" dropdown menu with the selected value "Terraform 60827 linux (amd64)". At the bottom of this section is a button "+ Add Installer". At the very bottom of the page are two buttons: "Save" (in blue) and "Apply" (in grey).

← → ↻ ⚠ Not secure 35.173.201.248:8080/manage/configureTools/ ☆

Jenkins / Manage Jenkins ▾ / Tools 🔍 ⚙️ 👤

+ Add Maven

Terraform installations

+ Add Terraform

⋮ Terraform ✕

Name

terraform

☒ Install automatically ?

⋮ Install from bintray.com ✕

Version

Terraform 60827 linux (amd64) ▾

+ Add Installer

Save Apply

3. Create a CI/CD pipeline for a Nodejs Application:

<https://github.com/betawins/Trading-UI.git>

1. Prerequisites

You need:

- AWS EC2 Linux server
- Jenkins
- Git
- Node.js + npm
- Docker
- GitHub repository
- Docker Hub account
- Jenkins Pipeline plugins
- Jenkins NodeJS plugin

The Jenkins NodeJS plugin can install/manage Node.js versions and make the configured Node.js installation available to Pipeline jobs.

Developer

|

| git push

↓

GitHub

|

| webhook / Jenkins trigger

↓

Jenkins

|

- |—— 1. Git Clone
- |—— 2. Install Node.js dependencies
- |—— 3. Lint / Test
- |—— 4. Build Node.js application
- |—— 5. Docker Build
- |—— 6. Docker Push
- |—— 7. Deploy



EC2 / Server



Trading UI Application

Jenkins recommends keeping the pipeline definition in a Jenkinsfile inside source control, which gives you Pipeline-as-Code.

The screenshot shows a terminal window within the AWS Management Console. The terminal output shows the following commands and results:

```
[root@ip-172-31-38-52 ~]# git clone https://github.com/betawins/Trading-UI.git
Cloning into 'Trading-UI'...
remote: Enumerating objects: 260, done.
remote: Counting objects: 100% (61/61), done.
remote: Compressing objects: 100% (14/14), done.
remote: Total 260 (delta 50), reused 47 (delta 47), pack-reused 199 (from 1)
Receiving objects: 100% (260/260), 246.56 KiB | 11.74 MiB/s, done.
Resolving deltas: 100% (116/116), done.
[root@ip-172-31-38-52 ~]# ll
total 0
drwxr-xr-x. 6 root root 186 Sep  8 17:37 Trading-UI
[root@ip-172-31-38-52 ~]# cd Trading-UI/
[root@ip-172-31-38-52 Trading-UI]# ls -la
total 568
drwxr-xr-x. 6 root root   186 Sep  8 17:37 .
dr-xr-xr-x. 5 root root   180 Sep  8 17:37 ..
drwxr-xr-x. 7 root root   147 Sep  8 17:37 .git
-rw-r--r--. 1 root root   291 Sep  8 17:37 'Install nodejs and npm using linux.txt'
-rw-r--r--. 1 root root   576 Sep  8 17:37 Jenkinsfile
-rw-r--r--. 1 root root  2881 Sep  8 17:37 README.md
drwxr-xr-x. 3 root root    87 Sep  8 17:37 coverage
-rw-r--r--. 1 root root 547386 Sep  8 17:37 package-lock.json
-rw-r--r--. 1 root root  1018 Sep  8 17:37 package.json
drwxr-xr-x. 2 root root   120 Sep  8 17:37 public
drwxr-xr-x. 5 root root 16384 Sep  8 17:37 src
[root@ip-172-31-38-52 Trading-UI]#
```

Below the terminal window, the instance details are shown:

i-0ccbde82075284734 (Jenkin-server)
PublicIPs: 100.26.192.107 PrivateIPs: 172.31.38.52

2. Understand the application first

Before creating Jenkins, clone the project manually on your server:

git clone https://github.com/betawins/Trading-UI.git

cd Trading-UI

Check the files:

```
ls -la
```

Most importantly, check:

```
cat package.json
```

You want to identify:

```
{  
  "scripts": {  
    "start": "...",  
    "test": "...",  
    "build": "..."  
  }  
}
```

The exact commands used by the pipeline should come from the project's package.json.

For example, if it contains:

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build"  
}
```

then your Jenkins build command will be:

```
npm run build
```

3. Install Jenkins

If Jenkins is already installed on your EC2 server, skip this section.

For Amazon Linux 2023, first install Java:

```
sudo dnf install java-21-amazon-corretto -y
```

Verify:

```
java -version
```

Then install Jenkins using the current Jenkins installation instructions appropriate to your environment.

Start Jenkins:

```
sudo systemctl enable jenkins
```

```
sudo systemctl start jenkins
```

Check:

```
sudo systemctl status jenkins
```

Open:

<http://<EC2-PUBLIC-IP>:8080>

Make sure the EC2 Security Group allows TCP 8080.

The screenshot shows the Jenkins web interface in a browser. The address bar displays "100.26.192.107:8080". The Jenkins logo is in the top left. On the left sidebar, there are links for "New Item", "Build History", "Project Relationship", and "Check File Fingerprint". Below these are two dropdown menus: "Build Queue" (showing "No builds in the queue.") and "Build Executor Status" (showing "0/2"). The main content area displays a table of build history. The table has columns for "S" (Status), "W" (Icon), "Name", "Last Success", "Last Failure", and "Last Duration". There are two builds listed: "java-working-app" and "java-working-app-parameterized". Both builds are successful, with the first build taking 26 seconds and the second taking 7.6 seconds. The footer of the page shows "REST API" and "Jenkins 2.568.3".

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	java-working-app	1 day 4 hr #1	N/A	26 sec
✓	☀	java-working-app-parameterized	1 day 3 hr #2	N/A	7.6 sec

4. Install Git

On Amazon Linux:

```
sudo dnf install git -y
```

Verify:

```
git --version
```

5. Install Docker

Install Docker:

```
sudo dnf install docker -y
```

Start Docker:

```
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

Verify:

```
docker --version
```

Give Jenkins permission to use Docker

This is important.

```
sudo usermod -aG docker jenkins
```

Restart Jenkins:

```
sudo systemctl restart jenkins
```

You may need to log out/reconnect to the server before testing the jenkins user's Docker access.

Test:

```
sudo -u jenkins docker ps
```

6. Configure Node.js in Jenkins

Go to:

Jenkins



Manage Jenkins



Plugins

Search for:

NodeJS

Install the **NodeJS plugin**.

The plugin supports configuring Node.js installations under Jenkins global tool configuration and exposing the selected installation to Pipeline jobs.

Then go to:

Manage Jenkins



Tools



NodeJS installations

Click:

Add NodeJS

For example:

Name: NodeJS-22

Select the option to install automatically if appropriate.

Save.



Plugins

📄 Updates

📁 Available plugins

⚙️ Installed plugins

⚙️ Advanced settings

🔍 NodeJS

📄 Install



Install	Name ↓	Released	Health
☒	NodeJS 1.6.6 npm NodeJS Plugin executes NodeJS script as a build step.	9 mo 28 days ago	97

The screenshot shows the Jenkins web interface at the URL `100.26.192.107:8080/manage/configureTools/`. The page title is "NodeJS installations". There is a button "+ Add NodeJS". Below it, a configuration card for "NodeJS" is shown. Inside the card, the "Name" field is set to "Node-22". The "Install automatically" checkbox is checked. Below this, there is a sub-section "Install from nodejs.org" with a "Version" dropdown menu set to "NodeJS 26.8.1". A note states: "For the underlying architecture, if available, force the installation of the 32bit package. Otherwise the build will fail". There is an unchecked checkbox for "Force 32bit architecture". At the bottom of the card, there is a section for "Global npm packages to install" with a text area containing the instruction: "Specify list of packages to install globally -- see npm install -g. Note that you can fix the packages version by using the syntax: package-name@version". At the bottom of the page, there are "Save" and "Apply" buttons.

← → ↻ ⚠ Not secure 100.26.192.107:8080/manage/configureTools/ ☆ Ⓐ ⋮

Jenkins / Manage Jenkins ▾ / Tools 🔍 ⚙️ 👤

NodeJS installations

+ Add NodeJS

⋮ NodeJS ✕

Name

Node-22

☒ Install automatically ?

⋮ Install from nodejs.org ✕

Version

NodeJS 26.8.1 ▾

For the underlying architecture, if available, force the installation of the 32bit package. Otherwise the build will fail

☐ Force 32bit architecture

Global npm packages to install

Specify list of packages to install globally -- see npm install -g. Note that you can fix the packages version by using the syntax: package-name@version

Save Apply

7. Install required Jenkins plugins

Go to:

Manage Jenkins

→ Plugins

Make sure you have the necessary plugins, particularly:

Pipeline

Git

GitHub Integration

NodeJS

Credentials Binding

The exact plugin set can vary depending on how you trigger the build and deploy.

8. Configure Docker Hub credentials

You need Docker Hub credentials so Jenkins can push the image.

Go to:

Jenkins

→ Manage Jenkins

→ Credentials

→ System

→ Global credentials

Click:

Add Credentials

Use:

Kind: Username with password

Username: <DockerHub username>

Password: <DockerHub access token>

ID: dockerhub-credentials

Prefer a Docker Hub access token rather than your normal Docker Hub password.

[Personal access tokens](#) / New access token

Copy access token

Use this token as a password when you sign in from the Docker CLI client. [Learn more](#)

Make sure you copy your personal access token now. Your personal access token is only displayed once. It isn't stored and can't be retrieved later.

Access token description

Jenkins-Terraform-CI-CD

Expires on

Never

Access permissions

Read & Write

To use the access token from your Docker CLI client:

1. Run

```
$ docker login -u rajkumari2010
```

Copy

2. At the password prompt, enter the personal access token.

```
dckr_pat_pjcKMIN3srgoUTRQezC0jNT03bM
```

Copy



Jenkins

/ Manage Jenkins



/ Credentials

/ System

/ Global



Global

+ Add Credentials

Credentials that should be available everywhere.



docker-terraform rajkumari2010/*****

dockerhub-terraform-credentials



9. Decide your Docker image name

For example:

rajkumari2010/trading-ui

The final image could be:

rajkumari2010/trading-ui:latest

For a better CI/CD approach, use the Jenkins build number:

rajkumari2010/trading-ui:15

This gives every build a unique image tag.

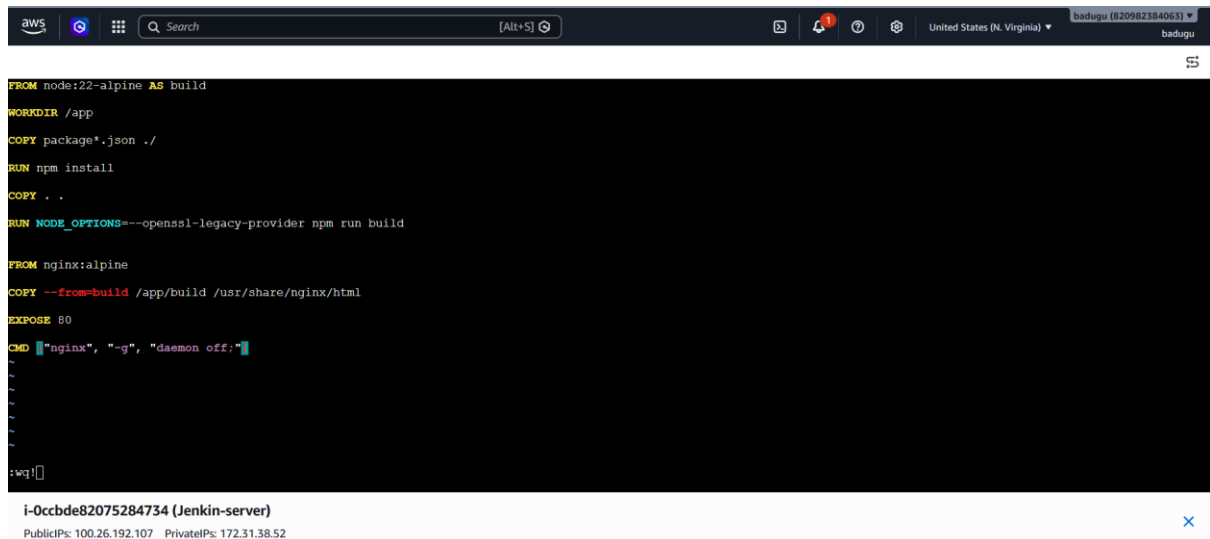
10. Create a Dockerfile

First determine whether this project already has a Dockerfile:

ls -la

If there isn't one, you need to decide how the Trading UI is served.

For a typical frontend application that builds into a dist directory, a common production approach is a multi-stage Dockerfile:

A screenshot of a terminal window showing a multi-stage Dockerfile. The terminal has a dark background with light-colored text. The Dockerfile content is as follows:

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN NODE_OPTIONS=--openssl-legacy-provider npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

The terminal window includes a top bar with the AWS logo, a search bar, and system information like "United States (N. Virginia)". At the bottom, it shows the container ID "i-0ccbde82075284734 (Jenkin-server)" and IP addresses.

docker build --no-cache -t trading-ui .



npm ci



source code



npm run build



dist/

Stage 2

Nginx



copy dist/



serve Trading UI

This is a **multi-stage Docker build**.

```
[root@ip-172-31-38-52 Trading-UI]# docker images
REPOSITORY          TAG         IMAGE ID      CREATED       SIZE
trading-ui           latest      930eedfe7ea9  6 minutes ago 72.7MB
rajkumari2010/trading-ui latest      930eedfe7ea9  6 minutes ago 72.7MB
[root@ip-172-31-38-52 Trading-UI]# docker run -d \
--name trading-ui \
-p 8081:80 \
rajkumari2010/trading-ui:latest
1353fe9c9d37c66cbe87a694f419a0231cbe7cad297be3862e3760908c5d54e
[root@ip-172-31-38-52 Trading-UI]# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED       STATUS       PORTS                               NAMES
1353fe9c9d37   rajkumari2010/trading-ui:latest     "/docker-entrypoint..." 3 minutes ago Up 3 minutes 0.0.0.0:8081->80/tcp, :::8081->80/tcp trading-ui
[root@ip-172-31-38-52 Trading-UI]#
```

← → ↻ Not secure 100.26.192.107:8081/#/ ☆ ⓘ ⋮

not found

InstaTrade – Invest And Grow With US Login

Login

Email Id should be in proper format

Email Id

Enter the email

password

Enter the password

Login

15. Create Jenkins Pipeline job

Go to:

Jenkins Dashboard



New Item

Enter:

Trading-UI-CI-CD

Select:

Pipeline

Click:

OK

16. Configure Pipeline from GitHub

Inside the Jenkins job:

Pipeline

Choose:

Definition:

Pipeline script from SCM

SCM:

Git

Repository:

<https://github.com/betawins/Trading-UI.git>

Branch:

*/main

Script Path:

Jenkinsfile

Then:

Save

Jenkins will load the Jenkinsfile from your repository. This is the recommended Pipeline-as-Code approach.

aws

1

United States (N. Virg

badugu (820982384063)

badugu

console.log
res true

at src/Components/Purchase/Purchase.js:156:21

console.log
validation response true

at src/Components/Purchase/Purchase.js:164:21

PASS src/__tests__/Purchase/Purchase.test.js
when the home component is called
✓ should render the render method (60 ms)
✓ should render the 1 button (10 ms)
✓ should have called handle click book function (25 ms)

Test Suites: 1 passed, 1 total
Tests: 3 passed, 3 total
Snapshots: 0 total
Time: 1.601 s
Ran all test suites matching /src\/__tests__\/Purchase\/Purchase.test.js/i.
Jest did not exit one second after the test run has completed.

This usually means that there are asynchronous operations that weren't stopped in your tests. Consider running Jest with `--detectOpenHandles` to troubleshoot this issue.
^C^C
[root@ip-172-31-38-52 Trading-UI]# npm test -- --watchAll=false

i-0ccbde82075284734 (Jenkin-server)

PublicIPs: 34.235.159.128 PrivateIPs: 172.31.38.52

X



```
t-renderer-shallow.development.js:794:32)
    at fn (node_modules/enzyme-adapter-react-16/src/ReactSixteenAdapter.js:636:55)
    at withSetStateAllowed (node_modules/enzyme-adapter-utils/src/Utils.js:99:18)
    at Object.render (node_modules/enzyme-adapter-react-16/src/ReactSixteenAdapter.js:636:20)
    at new ShallowWrapper (node_modules/enzyme/src/ShallowWrapper.js:397:22)
    at shallow (node_modules/enzyme/src/shallow.js:10:10)
    at Object.<anonymous> (src/__tests__/Header/Header.test.js:7:24)
```

PASS src/__tests__/Logout/Logout.test.js

Test Suites: 9 passed, 9 total
Tests: 13 passed, 13 total
Snapshots: 0 total
Time: 2.769 s, estimated 4 s
Ran all test suites.

Jest did not exit one second after the test run has completed.

This usually means that there are asynchronous operations that weren't stopped in your tests. Consider running Jest with `--detectOpenHandles` to troubleshoot this issue.

^C

[root@ip-172-31-38-52 Trading-UI]# npm run build

> bank@0.1.0 build

> react-scripts build

Creating an optimized production build...

i-0ccbde82075284734 (Jenkin-server)

PublicIPs: 34.235.159.128 PrivateIPs: 172.31.38.52



← → ↻ ⚠ Not secure 34.235.159.128:8080/job/Trading-UI/configure ☆ ⓘ ⋮

 **Jenkins** / Trading-UI ▾ / Configure 🔍 ⚙️ 👤

Repository URL ?

Credentials ?

▾ + Add

Advanced ▾

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

+ Add Branch

Save Apply

← → ↻ ⚠ Not secure 35.173.201.248:8080/job/java-working-app-parameterized/2/console 🔍 ☆ ⓘ ⋮

 **Jenkins** java-working-app-paramere... ▾ / #2 ▾ 🔍 ⚙️ 👤

```
[INFO]
[INFO] --- maven-compiler-plugin:3.1:testCompile (default-testCompile) @ hiring ---
[INFO] No sources to compile
[INFO]
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ hiring ---
[INFO] No tests to run.
[JENKINS] Recording test results
[INFO]
[INFO] --- maven-war-plugin:3.3.1:war (default-war) @ hiring ---
[INFO] Packaging webapp
[INFO] Assembling webapp [hiring] in [/var/lib/jenkins/workspace/java-working-app-parameterized/target/hiring]
[INFO] Processing war project
[INFO] Copying webapp resources [/var/lib/jenkins/workspace/java-working-app-parameterized/src/main/webapp]
[INFO] Building war: /var/lib/jenkins/workspace/java-working-app-parameterized/target/hiring.war
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ hiring ---
[INFO] Installing /var/lib/jenkins/workspace/java-working-app-parameterized/target/hiring.war to /var/lib/jenkins/.m2/repository/in/javahome/hiring/0.1/hiring-0.1.war
[INFO] Installing /var/lib/jenkins/workspace/java-working-app-parameterized/pom.xml to /var/lib/jenkins/.m2/repository/in/javahome/hiring/0.1/hiring-0.1.pom
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.617 s
[INFO] Finished at: 2026-09-07T14:15:00Z
[INFO] -----
Waiting for Jenkins to finish collecting data
[JENKINS] Archiving /var/lib/jenkins/workspace/java-working-app-parameterized/pom.xml to in.javahome/hiring/0.1/hiring-0.1.pom
[JENKINS] Archiving /var/lib/jenkins/workspace/java-working-app-parameterized/target/hiring.war to in.javahome/hiring/0.1/hiring-0.1.war
channel stopped
Finished: SUCCESS
```

← → ↻ ⚠ Not secure 34.235.159.128:8081/#/ ☆ ⓘ

not found

InstaTrade – Invest And Grow With US

Login

Login

Email Id

Enter the email

password

Enter the password

Login

5.Explain 10 Maven commands.

1. mvn --version

Checks the installed Maven version and environment.

```
mvn --version
```

Example output:

Apache Maven 3.8.4

Java version: 17

OS: Linux

Use: Verify whether Maven is installed correctly and which Java version Maven is using.

2. mvn clean

Deletes the project's previously generated target directory.

mvn clean

Use: Removes old compiled files before starting a fresh build.

Example:

project/

|—— src/

|—— pom.xml

|—— target/ ← removed by mvn clean

3. mvn compile

Compiles the application's source code.

mvn compile

Use: Converts Java source files (.java) into bytecode (.class).

Flow:

Java Source Code

↓

mvn compile

↓

Compiled .class files

↓

target/classes/

4. mvn test

Runs the unit tests.

mvn test

Use: Checks whether the application functionality is working as expected.

Flow:

Source Code



Compile



Unit Tests



PASS / FAIL

In Jenkins, you commonly see:

sh 'mvn test'

5. mvn package

Compiles the code, runs tests, and creates a distributable package such as a **JAR** or **WAR**.

mvn package

For example:

target/

└── myapp-1.0.jar

or:

target/

└── myapp-1.0.war

Use: Create the application artifact that can later be deployed.

6. mvn install

Builds the project and installs the generated artifact into the **local Maven repository**.

mvn install

The local repository is normally:

~/.m2/repository/

Flow:

Source Code

↓

Compile

↓

Test

↓

Package

↓

Install

↓

~/.m2/repository/

Use: When another local Maven project needs to use this project as a dependency.

7. mvn clean package

Performs a clean build and creates the JAR/WAR package.

mvn clean package

It essentially performs:

clean

↓

compile

↓

test

↓

package

Use: Very common in Jenkins pipelines because it avoids using old build files.

Example:

```
stage('Maven Build') {  
    steps {  
        sh 'mvn clean package'  
    }  
}
```

8. mvn dependency:tree

Displays the project's dependency hierarchy.

mvn dependency:tree

Example:

```
com.example:myapp  
├── org.springframework:spring-core  
├── mysql:mysql-connector  
└── org.apache.commons:commons-lang3
```

Use: Find which dependencies are being used and troubleshoot **dependency conflicts**.

9. mvn validate

Checks whether the Maven project is properly configured.

mvn validate

It validates things such as the pom.xml structure and required project information.

Use: Quickly check whether the Maven project configuration is valid before performing a complete build.

10. mvn deploy

Builds the project and uploads the artifact to a **remote Maven repository**, such as Nexus or another artifact repository.

mvn deploy

Typical DevOps flow:

Developer



GitHub



Jenkins



Maven Build



JAR / WAR



Nexus Repository



Deployment

For example, Jenkins can use:

```
stage('Maven Deploy') {  
    steps {  
        sh 'mvn clean deploy'  
    }  
}
```


}

Quick interview table

Command	Main Purpose
mvn --version	Check Maven version
mvn clean	Delete old target files
mvn compile	Compile source code
mvn test	Run unit tests
mvn package	Create JAR/WAR
mvn install	Install artifact into local .m2 repository
mvn clean package	Clean + test + create JAR/WAR
mvn dependency:tree	Display dependency hierarchy
mvn validate	Validate Maven project
mvn deploy	Upload artifact to remote repository